분야 AI/SW 심화 구분 AI 수학 학습시간 60시간 필수/선택 필수

AI가 어떻게 학습하는지 수학으로 직접 풀어보기

문제기술

기술적 설명



1. 미션 소개

선형대수, 미적분, 확률통계는 AI 모델이 데이터를 처리하고 학습하는 과정의 수학적 기반이다. 신경망의 각 층에서 일어나는 연산은 행렬 곱셈이며, 가중치를 업데이트하는 과정은 경사하강법이라는 최적화 알고리즘을 따른다. 이 미션에서는 수학적 계산과 구현은 NumPy로, 시각화는 Matplotlib/Seaborn을 사용하여 행렬 변환의 기하학적 의미를 시각화하고, 고유값 분해를 직접 구현하며, 연쇄 법칙 기반의 역전파를 수식으로 유도한다. 또한 경사하강법의 수렴 과정을 등고선 위에서 시각화하고, 손실 함수가 확률 분포와 어떻게 연결되는지 탐구한다. 이 경험은 이후 딥러닝 미션에서 AutoGrad 엔진을 구현할 때 수학적 기반이 되며, 모델의 동작 원리를 해석하고 문제를 진단하는 능력을 갖추게 한다.

2. 최종 결과물

다음 내용이 모두 포함된 **GitHub 레포지토리(Repository) URL**을 제출한다.

1. 선형대수 모듈 (src/linear_algebra.py)
    - 행렬 변환(회전, 스케일링, 전단) 시각화 함수
    - Power Iteration으로 최대 고유값과 대응 고유벡터를 계산하는 함수
    - SVD 기반 이미지 압축/복원 함수
2. 미적분 모듈 (src/calculus.py)
    - 수치 미분 함수 (중심차분법 기반)

- Gradient 시각화 함수

3. 역전파 유도 노트북 (notebooks/backprop_derivation.ipynb)
    - 2층 신경망 순전파/역전파 수식 유도
    - 필수 고정 예제의 손계산 과정 (중간 체크포인트 + shape 표기 포함)
    - NumPy 코드 검증 결과
4. 최적화 모듈 (src/optimizer.py)
    - VanillaGD, Momentum 옵티마이저 클래스
    - 수렴 경로 시각화 함수
5. 확률/통계 노트북 (notebooks/probability_loss.ipynb)
    - 확률 분포 시각화 (정규분포 PDF 2종, 베르누이 PMF 2종)
    - MSE-MLE, Cross-Entropy-MLE 수식 연결 설명
6. README.md
    - 프로젝트 개요, 실행 방법, 주요 결과 요약

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 행렬 곱셈이 공간을 어떻게 변형시키는지, 행렬식이 면적 변화율과 어떻게 연결되는지 설명할 수 있다
- 고유값과 고유벡터가 선형 변환에서 어떤 의미를 가지는지, SVD가 데이터 압축에 어떻게 활용되는지 이해한다
- 연쇄 법칙을 사용하여 2층 신경망의 역전파 수식을 유도하고, 각 가중치에 대한 기울기를 손계산할 수 있다
- MSE와 Cross-Entropy 손실 함수의 확률적 의미를 구분하고, MLE와의 연결을 수식으로 설명할 수 있다
- Learning Rate와 Momentum이 경사하강법 수렴에 미치는 영향을 실험 결과와 함께 설명할 수 있다

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 선형대수: 행렬 변환과 분해

- 단위 원에 회전 행렬 R(θ)를 적용하고 변환 전후를 하나의 Figure에 겹쳐 출력한다
- 단위 원에 스케일링 행렬 S(2, 0.5)를 적용하고 타원 형태의 결과를 출력한다
- 단위 원에 전단 행렬 Sh(k)를 적용하고 기울어진 형태의 결과를 출력한다
- 행렬식 det(A)를 계산하고, 변환 후 도형의 면적 비율과 비교하여 오차가 1% 이내임을 확인한다
- Power Iteration 알고리즘을 구현하여 최대 고유값을 계산한다
- 직접 계산한 고유값과 np.linalg.eig 결과를 비교하여 오차가 5% 이내임을 확인한다
- SVD를 활용하여 이미지를 압축하고, k=10, 50, 100일 때 복원 결과를 하나의 Figure에 비교 출력한다

2. 미적분: Gradient 시각화

- $f(x)=x^2$에서 x=3일 때 수치 미분을 계산하고, 해석적 미분 결과(6)와 비교하여 오차가 1e-4 이내임을 확인한다
- 2변수 함수 $f(x,y)=x^2+y^2$의 등고선 그래프를 그린다
- 등고선 위 여러 점에서 Gradient 방향을 화살표로 표시한다
- Gradient 화살표가 등고선에 수직임을 시각적으로 확인한다

3. 역전파: 연쇄 법칙 수식 유도

- 2층 신경망(입력 2개 → 은닉 2개 → 출력 1개)의 순전파 수식을 노트북에 작성한다
- 출력층에서 입력층 방향으로 연쇄 법칙을 적용하여 역전파 수식을 단계별로 유도한다
- 구체적인 숫자 예제로 순전파를 손계산한다
    - 입력: x = [1, 0]
    - 가중치 1: W1 = [[0.1, 0.2], [0.3, 0.4]]
    - 편향 1: b1 = [0, 0]
    - 가중치 2: W2 = [0.5, 0.6]
    - 편향 2: b2 = 0
    - 활성화 함수: Sigmoid
    - 손실 함수: Binary Cross-Entropy (BCE) L = -(y·log(ŷ) + (1-y)·log(1-ŷ))
    - 정답: y_true =
- 순전파 중간값 : z1, a1, z2, y_pred 각각을 기록한다

- 역전파 필수 유도 : ∂L/∂y_pred, ∂L/∂z2, ∂L/∂W2, ∂L/∂a1, ∂L/∂z1, ∂L/∂W1 모두 유도한다
- 각 단계에서 텐서의 shape을 반드시 표기한다 (예: W1: (2,2), x: (2,), z1: (2,) 등)
- NumPy 코드로 동일한 계산을 수행하고, 손계산 결과와 소수점 4자리까지 일치함을 확인한다

4. 최적화: 경사하강법 시각화

- Vanilla GD를 구현하고 $f(x,y)=x^2+y^2$에 적용한다
- lr=0.1, 100회 반복 후 (x,y)가 원점 근처(반경 0.1 이내)에 도달함을 확인한다
- 수렴 경로를 등고선 위에 점선으로 출력한다
- lr=0.5 이상에서 최적화 경로가 발산함을 그래프로 확인한다
- Momentum(beta=0.9)을 구현하고 Vanilla GD와 수렴 경로를 비교한다
- 두 옵티마이저의 수렴 경로를 하나의 Figure에 겹쳐 출력한다
- 타원형 함수 $f(x,y)=x^2+10y^2$에서도 Vanilla GD와 Momentum의 수렴 경로를 비교하여 Momentum의 이점을 확인한다

5. 확률/통계: 손실 함수의 수학적 기반

- 정규분포 N(0,1)과 N(2,0.5)의 PDF 곡선을 하나의 Figure에 비교 출력한다
- 베르누이 분포 B(0.3)과 B(0.7)의 PMF 막대그래프를 하나의 Figure에 비교 출력한다
- 소프트맥스 함수를 구현하고, 출력 벡터의 합이 1.0과의 차이가 1e-6 이내임을 확인한다
- "MSE 최소화 = 정규분포 가정 하의 MLE"임을 수식으로 유도하여 노트북에 작성한다
- "Cross-Entropy 최소화 = 베르누이/카테고리 분포 가정 하의 MLE"임을 수식으로 유도하여 노트북에 작성한다

5. 보너스 과제 (선택)

1. Adam 옵티마이저 구현

- Momentum과 RMSProp을 결합한 Adam 옵티마이저를 구현한다
- Vanilla GD, Momentum, Adam의 수렴 속도를 비교한다
- 배움 포인트 : 적응적 학습률의 효과를 이해한다

2. Newton Method 구현

- Hessian(2차 미분)을 활용한 Newton Method를 구현하고 경사하강법과 비교한다
- 배움 포인트 : 2차 최적화 기법의 장단점을 이해한다

3. 정보 이론 구현

- 엔트로피, KL-Divergence, Cross-Entropy를 직접 구현한다
- 배움 포인트: 정보 이론이 머신러닝에서 활용되는 원리를 이해한다

[참고 사항] 데이터셋 확보 가이드

행렬 변환 실험

- 단위 원: theta = np.linspace(0, 2*np.pi, 100)으로 생성
- 격자: np.meshgrid로 2D 점 집합 생성

이미지 압축 실험

- 64x64 이하의 Grayscale 이미지 사용
- 직접 촬영하거나 공개 이미지 활용

최적화 테스트 함수

- 2차 함수: $f(x, y) = x^2 + y^2$
- 타원형 함수: $f(x, y) = x^2 + 10y^2$ (Momentum 효과 관찰용)

2층 신경망 손계산 예제

- 구조: 입력 2개 → 은닉 2개 → 출력 1개
- 가중치: 소수점 1자리의 간단한 값 (예: 0.1, 0.2, 0.3)
- 활성화 함수: Sigmoid

개발환경

6. 개발 환경

- Python 3.8 이상
- 권장 버전 : NumPy >= 1.21, Matplotlib >= 3.4
- 재현성 : np.random.seed(42)로 시드를 고정한다

- 시각화 결과는 outputs/ 디렉토리에 PNG로 저장한다

제약조건

7. 제약 사항

허용 라이브러리 (구현/시각화용)

- NumPy : 모든 수학적 계산과 구현에 사용
- Matplotlib, Seaborn : 시각화에만 사용

허용 라이브러리 (검증 목적 전용)

- linalg.eig : Power Iteration 결과와 비교하는 검증 코드에서만 사용한다. 고유값 계산 자체에는 사용하지 않는다.

금지 사항

- PyTorch, TensorFlow, JAX 등 자동 미분 라이브러리를 사용하지 않는다
- scikit-learn의 PCA, 최적화 함수를 사용하지 않는다

코드 품질

- 모든 클래스와 함수에 Docstring을 작성한다
- 수학적 수식을 주석 또는 Markdown으로 명시한다
- requirements.txt 에 사용 라이브러리와 버전을 명시한다

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 구조는 달라도 된다.

- 행렬 변환 시각화 예시 :

```
CSS
[스케일링 행렬 S(2, 0.5) 적용]
변환 전: 단위 원 (파란색)
변환 후: 타원 (빨간색, 가로 2배, 세로 0.5배)
```

```
det(S) = 1.0, 면적비 = 1.0, 오차 = 0%
```

- 고유값 분해 예시 :

```
CSS
[Power Iteration vs np.linalg.eig]
행렬 A = [[4, 1], [1, 3]]  ← 대칭 행렬
Power Iteration (수렴 51회, tol=1e-6): λ=4.6180, v=[0.8507, 0.5257]
np.linalg.eig: λ=4.6180, v=[0.8507, 0.5257]
오차: 0.0% (5% 이내) → 검증 완료
```

- 역전파 손계산 예시 :

```
CSS
[2층 신경망: 입력(2) → 은닉(2) → 출력(1)]

순전파 (shape 명시):
  x:      (2,)   = [1, 0]
  W1:     (2,2)  = [[0.1, 0.2], [0.3, 0.4]]
  z1:     (2,)   = W1 @ x = [0.1, 0.3]
  a1:     (2,)   = sigmoid(z1) = [0.5250, 0.5744]
  W2:     (2,)   = [0.5, 0.6]
  z2:     scalar = W2 @ a1 = 0.6071
  y_pred: scalar = sigmoid(z2) = 0.6473

역전파 (y_true=1, BCE Loss):
  ∂L/∂y_pred: scalar = -(1/0.6473) = -1.5449
  ∂L/∂z2:     scalar = y_pred - y_true = -0.3527   ← 체크포인트
  ∂L/∂W2:     (2,)   = ∂L/∂z2 * a1 = [-0.1852, -0.2026]
  ∂L/∂a1:     (2,)   = ∂L/∂z2 * W2 = [-0.1764, -0.2116]
  ∂L/∂z1:     (2,)   = ∂L/∂a1 * σ'(z1) = [-0.0440, -0.0513]   ← 체크포인트
  ∂L/∂W1:     (2,2)  = outer(∂L/∂z1, x) = [[-0.0440, 0], [-0.0513, 0]]

NumPy 검증: 손계산 결과와 소수점 4자리까지 일치
```

- MSE-MLE 연결 수식 예시 :

```
CSS
[MSE 최소화 = 정규분포 MLE]

가정: y = f(x) + ε, ε ~ N(0, σ²)
따라서: p(y|x) = (1/√(2πσ²)) * exp(-(y - f(x))² / 2σ²)

최대우도추정(MLE):
  argmax Π p(yᵢ|xᵢ) = argmax Σ log p(yᵢ|xᵢ)
  = argmax Σ [-(yᵢ - f(xᵢ))² / 2σ² - log(√(2πσ²))]
  = argmin Σ (yᵢ - f(xᵢ))²  (상수 제거)
  = argmin MSE

결론: MSE를 최소화하는 것은 정규분포 가정 하에서 MLE와 동일하다.
```

- 옵티마이저 비교 예시 :

```
CSS
[f(x,y)=x²+y², 초기점 (5, 5), lr=0.1]
100회 반복 후:
  Vanilla GD: (0.08, 0.08) - 원점 도달
  Momentum:   (0.02, 0.02) - 더 빠르게 수렴

[f(x,y)=x²+10y², 초기점 (5, 5), lr=0.05]
  Vanilla GD: 지그재그 경로로 느린 수렴
  Momentum:   부드러운 곡선으로 빠른 수렴
```